PyGame Car Crash Tutorial - Part 7

Contents

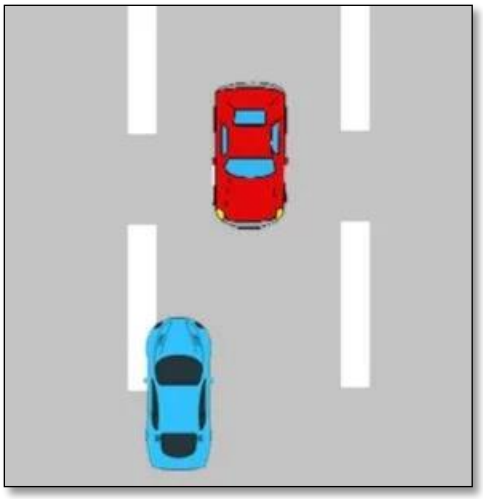
PyGame Car Crash Tutorial - Part 7 1
Preview of the Game 1
config.py 2
enemy.py 2
car_crash_7.py 4
Sound 5
What's Next? 10
Assignment Submission 10

Time required: 30 minutes

Preview of the Game

Here's a sneak peak of the game that we are going to work on.

CarCrashDemo Video



Car Crash is simple arcade type game. The object is to move your blue car back and forth to avoid the oncoming red cars.

Let's finish up our car game with sounds, fonts, and a scoring system.

config.py

```python
"""
    Filename: config.py
    Author:
    Date:
    Purpose: Global variables and constants for the entire program
"""
# Import config module into all other modules

# Setup global constants and variables for screen size
WIDTH = 400
HEIGHT = 600

# Global variable for speed across screen
SPEED = 4

# Constant for how much the speed increases each enemy car pass
SPEED_INCREASE = .4

BLACK = (0, 0, 0)
```

enemy.py

To keep score, initialize a score variable in the enemy.py class.

```python
class Enemy(pygame.sprite.Sprite):
    """Define the enemy class and methods"""
# ------------------------ INITIALIZE ENEMY OBJECT ------------------------#

    def __init__(self):
        self.score = 0
        """Construct an enemy object from Sprite class"""

        # Call the constructor of the superclass (pygame.sprite.Sprite)
        super().__init__()

        self.score = 0
        self.speed = config.SPEED

        # Load enemy car image from file into a variable
        self.image = pygame.image.load(
            "./assets/enemy.png").convert_alpha()
```

We keep track of how many times the Enemy gets to the bottom of the screen. Each time this happens, the player gets one more point.

```python
    # -------------------------- UPDATE ----------------------------------#
    def update(self):
        # Move the sprite down SPEED pixels at a time
        self.rect.move_ip(0, self.speed)

        # When the top of the sprite reaches the botton of the surface
        if (self.rect.top > config.HEIGHT):
            # Get a random location 40 pixels away from left and right.
            x = randint(40, config.WIDTH - 40)

            # Move car above the program window
            y = -120

            # Move car to beginning position
            self.rect.center = (x, y)

            # Increase speed each time the enemy car starts at the top
            self.speed += config.SPEED_INCREASE

            # Increment score every time the player dodges an oncoming car
            self.score += 1
```

car_crash_7.py

We are going to add another library to display our game over screen.

```
pip install pygame-menu
```

```python
"""
    Filename: car_crash_7.py
    Author:
    Date:
    Purpose: Add scoring, sound and a game over menu
"""
# pip install pygame-ce
# Import modules
import pygame
# pip install pygame-menu
import pygame_menu as pm
from sys import exit
from time import sleep
# Import our game classes
import config
import player
import enemy
```

Sound

```python
class CarCrash:
    def __init__(self):
        # Pre initalize mixer with larger buffer size for better performance
        pygame.mixer.pre_init(
            44100,  # frequency (Hz)
            16,     # bit depth
            2,      # number of channels, 1 mono, 2 stereo
            4096    # buffer size, larger to optimize music playback.
        )

        # Initialize the pygame library
        pygame.init()
```

```python
        # Optimize game by only allowing these events to be captured
        pygame.event.set_allowed(
            [pygame.QUIT, pygame.KEYDOWN, pygame.KEYUP]
        )

        # Set window icon
        window_icon = pygame.image.load('./assets/car.ico')
        pygame.display.set_icon(window_icon)

        # Create the player and enemy sprites
        self.create_sprites()

        # Create System font object for score
        self.font_small = pygame.font.SysFont("arialblack", 20)

        # Load sound file into memory
        pygame.mixer.music.load('./assets/background_music.wav')

        # Set volume to 30%, range from 0.0 (mute) to 1.0 (full volume)
        pygame.mixer.music.set_volume(0.3)

        # Play in a loop until stopped
        pygame.mixer.music.play(-1)
```

Add the display_game_over method.

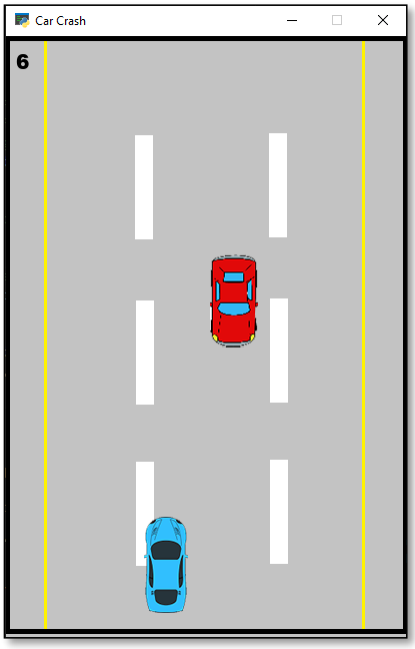
```python
# ------------------------- DISPLAY GAME OVER ----------------------------#
    def display_game_over(self):
        """Display game over on top of the stopped game"""
        # Stop background sound
        pygame.mixer.music.stop()

        # Play crash sound
        crash = pygame.mixer.Sound("assets/crash.wav")
        crash.play()
        crash.set_volume(0.5)

        # Wait 1 second
        sleep(1)

        # Define a menu object for the game over screen
        game_over = pm.Menu(
            title="Game over",      # Set title menu to "Game over"
            width=config.WIDTH,     # Set to width of game surface
            height=config.HEIGHT,   # Set to height of game surface
            # Set the theme of the menu to an orange color scheme
            theme=pm.themes.THEME_ORANGE
        )

        # Display final score
        game_over.add.label(f"Score: {self.enemy_sprite.score}")

        # Add label to provide space between buttons
        game_over.add.label("")

        # Add a button to the game over menu for exiting the game
        game_over.add.button(
            title="Play Again?",    # Button text
            action=main             # Call main() to start over
        )

        # Add label to provide space between buttons
        game_over.add.label("")

        # Add a button to the game over menu for exiting the game
        game_over.add.button(
            title="Exit",           # Button text
            action=pm.events.EXIT   # Exit the game when clicked
        )

        # Run the main loop of the game over menu on the specified surface
        game_over.mainloop(self.surface)
```

There are different themes you can choose for the game_over object. This example uses THEME_ORANGE. You can use any of the following to customize your menu.

```
THEME_BLUE
THEME_DARK
THEME_DEFAULT
THEME_GREEN
THEME_ORANGE
THEME_SOLARIZED
```

```python
# ---------------------- GAME LOOP -------------------------------------#
    def game_loop(self):
        """Start the infinite Game Loop"""
        while True:
            self.check_events()
            self.check_collision()

            # --------------- DRAW ON BACKBUFFER ------------------------#
            # Draw everything on the backbuffer first
            # Fill the surface with the background image loaded earlier
            self.surface.blit(self.background, (0, 0))

            # ------------- UPDATE AND DRAW SPRITES ---------------------#
            # Run the update method on all sprites
            self.all_sprites.update()

            # Draw all sprites on the surface
            self.all_sprites.draw(self.surface)

            # Render score before drawing it on the surface
            self.score = self.font_small.render(
                str(self.enemy_sprite.score), True, config.BLACK
            )

            # Draw score on the surface
            self.surface.blit(self.score, (10, 10))

            # --------------- UPDATE SURFACE ----------------------------#
            # From backbuffer, update Pygame display to reflect any changes
            pygame.display.update()

            # Cap game speed at 60 frames per second
            self.clock.tick(60)


def main():
    # Create game instance
    car_crash = CarCrash()
    # Start the game
    car_crash.game_loop()


main()
```

Example run:



What's Next?

There is much more that can be done with this game. Here are some ideas for you to practice and implement on your own.

- Keep track of the score between games.
- Multiple enemies spawning after set periods of time. (Similar to how we increased speed after a set period of time)
- Add some additional audio to the game, such as movement sounds (audio that plays when you move the character)
- Add the concept of multiple Lives or a Health bar.
- Variations in the shape and size of the "enemies".

Assignment Submission

Zip up the program files folder and submit in Blackboard.